Multi-Task Learning for Stock Price Prediction

Computational Intelligence (CIS*4780) - March 2026

Stella Rovazzi
srovazzi@uoguelph.ca

Matthew Lawford
mlawford@uoguelph.ca

Julia Horner
jhorne03@uoguelph.ca

Abstract

Insert abstract here

I. Introduction

Insert introduction here

II. Motivation and Objectives

Motivation

Comparing and exploring multi-tasking

Objectives

Figure out if multi-tasking stock data is better

III. Methodology

A. Data

B. Algorithms

LSTMs: explination To implement this, we used pytorch.

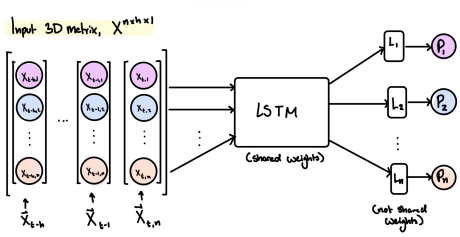

Fig. 1. Diagram of hard parameter sharing and inputs for an LSTM Multi-Tasking model. (Drawn by group).

Multi-Tasking Extra Trees (or Extremely Randomized Trees): Extra Trees (ET) is a tree ensemble method like Random Forests, but it produces more decorrelated trees. It uses the entire data set to build decision trees, rather than bootstrapping as in random forests. It also differs in its threshold selection, which randomly selects where to split the feature rather than using an optimization method. It still chooses m_{try} number of random features to choose the feature splits to compare, where $m_{try} = \lfloor \frac{p}{3} \rfloor$ and p is the number of features. There are two relevant extensions to this algorithm that make it similar to our LSTM hard parameter multi-tasking [1].

The first is introducing multi-output, which essentially means the loss function is the summation of sample error and task outputs. This effectively makes each node split consider the error of the candiate splits' effect across all tasks [1]. The second is introducing task-focused splits, or the ability to make decisions based on what the task is being predicted. To do this, a random group of m_{try} features and S random possible subsets of tasks are selected (S$\epsilon\mathbb{Z} < T$ randomly selected). Each feature split is randomized, and the multi-output error is calculated. Then, f_k (Equation 1) is calculated for each candidate task subset or feature; this is the prediction. The error is calculated from the prediction and used to choose the best split at the node [1].

$$f_k = \frac{\sum_{\nu \epsilon I_k} y_\nu + \alpha \frac{1}{\lfloor I \rfloor} \sum_{w \epsilon I} y_w}{\lfloor I_k \rfloor + \alpha} \tag{1}$$

"I denotes the set of sample indexes at the current node, I_k the set of sample indexes of the task k at the current node and α is a regularization parameter weighting the influence of the tasks with default value 1. Then, the task cut point is randomly selected from the interval $(\min_k f_k, \max_k f_k)$" - Schmid et. al [1]

To implement this method, we used R, since there is a pre-made R package for multi-tasking tree enssemblers. It is technically archived in newer versions of R, but can be downloaded here. This was decided because the paper explaining the metholody used this package [1].

C. Metrics and Evaluation

Rolling Cross-Validation:

Mean Absolute Error (Criterion and Metric):

D. Preprocessing

Misaligned Dates: Due to different market holidays, some days in the data included both null values and recorded prices. The days with null values were removed to keep the dataset consistent. Dropping a few days of data should not obscure the stocks' growth patterns and relationships.

Logarithmic Scale: The data was scaled logarithmically. This was to ensure stable predictions by reducing effect of outliers on the model and to account for the exponential patterns in each stock's data. The predictions become more stable because most analysis techniques assume additive data, including machine learning. [2].
The scaling affect is shown in appendix A, the medium stock ACIW is on a similar scale to the large stock AAPL after logging the data.

LSTMs: Pytorch requires LSTM inputs to be 3D tensors [3]. The data looks at stocks, with the only attribute being the stock price. Referring to figure 1, the matrix on the left would be an example 3D tensor input, with each vector $\vec{x}$ representing all the stocks of interest at the given t. There is no third dimension depicted, as the only attribute of each stock of interest is the given price at time t.
The times within the input are $\vec{x_{t,n}}$ and the lags of $\vec{x_{tn}}$, which represent our rolling window, since $\vec{x_{t+1}}$ should use all the previous lags for its prediction.

Extra Trees: To turn the stock price data into an acceptable input for a tree ensemble, the lags must be added manually. Lags are data from a previous date; for example, lag 1 at time t is the data point at time $t-1$ [4]. Each datapoint will then have 61 attributes: 1 representing time t's price and 60 representing the 60 associated lags [5]. Days without all 60 previous lags have been dropped.

E. Building the Models

LSTMs: describe the things we need to tune? And how single vs mutli looks diffrent (not what we chose just what we did method wise). Why chosen hyper parameters, why others not chosen.

Extra Trees: describe the things we need to tune? And how single vs mutli looks diffrent (not what we chose just what we did method wise)

IV. Results

add pics and stuff

A. Single-Task LSTM

Hyperparameters:
Algorithm Performances:

B. Multi-Task LSTM

Hyperparameters:
Algorithm Performances:

C. Single-Task Extra Trees

For single-task extra trees, IBM and Blackberry (BB.TO) stocks were evaluated. The plots used to tune the hyperparmeter are in B.

Hyperparameters: Blackberry had the smallest validation MAE with terminal node size 3 B4, 8 random cuts B6, and m_try size $\sqrt{p}$ B5. IBM Had the smallest validation MAE with terminal node size 3 B1, 2 random cuts B3, and m_try size $p/3$ B2.

Algorithm Performances: Blackberry's evaluation MAE was 0.02308594 for single-task ET. IBM's evaluation MAE was 0.01502721 for single task ET.

D. Multi-Task Extra Trees

Hyperparameters: Tuning showed 3 nodes and 5 nodes had almost equivalent MAE, so we chose 3 nodes.

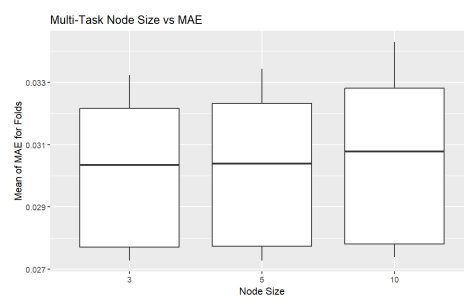

Fig. 2. Tuning Multi-Task Extra Trees node size

Tuning showed $p/3$ was the best for multi-tasking m_try.

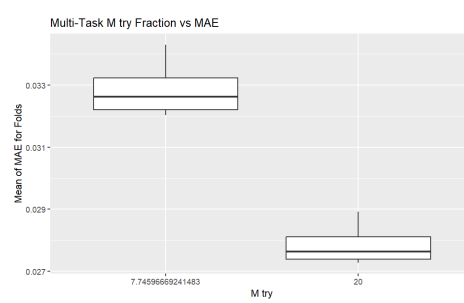

Fig. 3. Tuning Multi-Task Extra Trees m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

Tuning showed 10 random cuts would be ideal for multi-tasking.

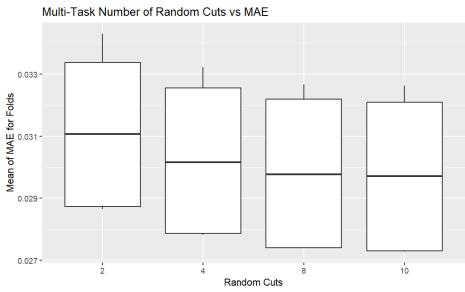

Fig. 4. Tuning Multi-Task Extra Trees number of random cuts

Algorithm Performances: Below is a table that shows the MAE of evaluation for each stock prediction in the multi-tasking extra trees model:

Task	Mean of MAE for Evaluation Window
AAPL	0.010017864
ACIW	0.013566978
AMD	0.031279329
BB.TO	0.015650922
BDC	0.017563812
BELF	0.019272468
BLKB	0.013493255
BMI	0.011254196
CLS.TO	0.012401716
CRM	0.013950118
CSCO	0.009417804
DSG.TO	0.011196125
GIB-A.TO	0.009294681
IBM	0.011959280
INTC	0.009181391
KXS.TO	0.017382097
MSFT	0.009562804
NOVT	0.013969065
NTCT	0.015411194
OLED	0.024107319
ORCL	0.008827831
OTEX.TO	0.009609518
PLAB	0.015337510
SLAB	0.014071373
UCTT	0.023878160
VSAT	0.012968538

V. Discussion

A. Findings

B. Interpretation of Results

1) Single vs Multi:

2) LSTM MTL vs ExtraTrees MTL:

C. Advantages and Disadvantages of Multi-Tasking

VI. Conclusion

A. Future Works

References

[1] L. Schmid, A. Gerharz, A. Groll, and M. Pauly, "Tree-based ensembles for multi-output regression: Comparing multivariate approaches with separate univariate ones," *Computational Statistics & Data Analysis*, vol. 179, p. 107628, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167947322002080

[2] "Decomposition of time series," 2025. [Online]. Available: https://medium.com/@m.capurso/decomposition-of-time-series-071ce7b7a5a0

[3] "Sequence models and long short-term memory networks," 2017. [Online]. Available: https://docs.pytorch.org/tutorials/beginner/nlp/sequence_models_tutorial.html

[4] J. Slater, "Time series analysis (stat*4360)," University of Guelph, 2025.

[5] I. P. Carrascosa, "Forecasting the future with tree-based models for time series," 2025. [Online]. Available: https://machinelearningmastery.com/forecasting-the-future-with-tree-based-models-for-time-series/

APPENDIX

A. Raw Data

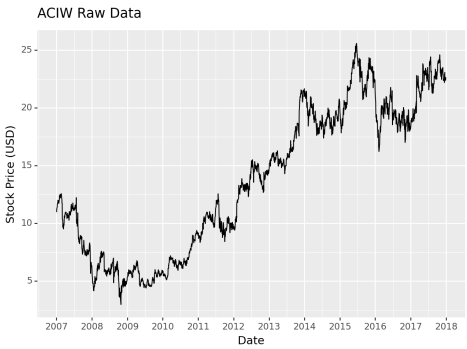

Fig. A1. Raw price data of ACIW Stock

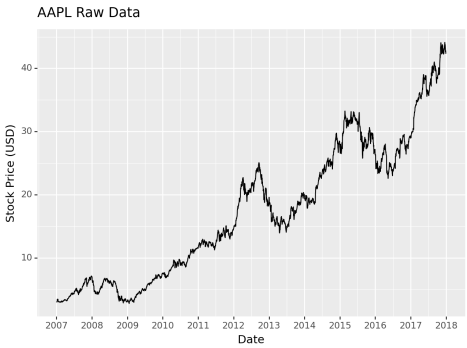

Fig. A2. Raw price data of APPL Stock

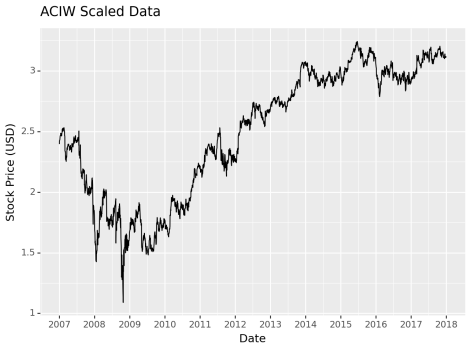

Fig. A3. Logged price data of ACIW Stock

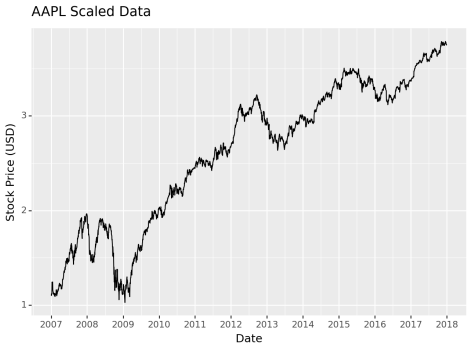

Fig. A4. Logged price data of APPL Stock

B. Single-Task Extra Trees Tuning

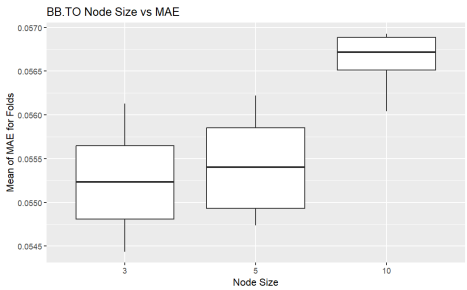

Fig. B1. Tuning Blackberry node size

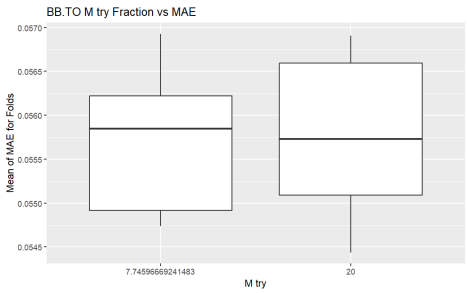

Fig. B2. Tuning Blackberry m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

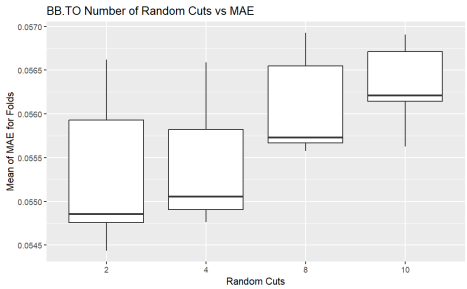

Fig. B3. Tuning Blackberry number of random cuts

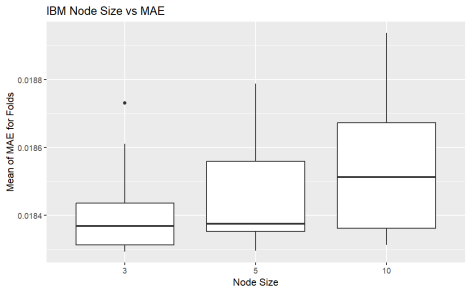

Fig. B4. Tuning IBM node size

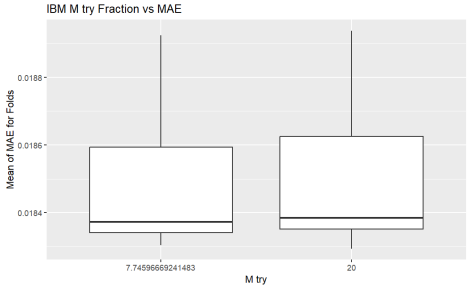

Fig. B5. Tuning IBM m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

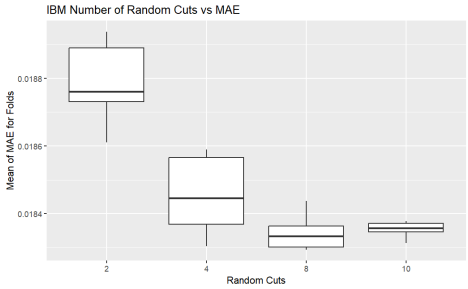

Fig. B6. Tuning IBM number of random cuts